

HELUT: Homomorphic Edge Look-Up Tensors

Compiling Gate-Level Netlists into Exact $\mathbb{Z}/2^{32}\mathbb{Z}$
Tensor Graphs on Apple Silicon

Digital Defiance

HELUT Project Report — systems prototype, measured results, explicit threat model

August 2026

Abstract

We present **HELUT** (Homomorphic Edge Look-Up Tensors), a systems prototype that compiles Yosys gate-level netlists into a single `MetalPerformanceShadersGraph` (`MPSGraph`) and evaluates every LUT as a dense negacyclic matrix–vector product over $\mathbb{Z}/2^{32}\mathbb{Z}$. The design deliberately avoids floating-point approximations and proprietary FHE runtimes: torus arithmetic is realized by native `UInt32` wraparound on Apple Silicon, and programmable look-ups are realized by expanding key polynomials into explicit Negacyclic Toeplitz matrices that the GPU/ANE evaluates via broadcast multiplication and `reductionSum`.

Sequential logic is emulated by a host clock loop that feeds D-flip-flop Q placeholders from the previous tick’s Q_{next} tensors, including enable and synchronous-reset semantics required by real Yosys techmaps. As a capstone, we compile a PicoRV32 RISC-V core (4785 LUTs, 1565 DFFs) and step it for ten mock-encrypted clock cycles under a scripted `resetn` sequence. On the evaluation machine, a debug end-to-end boot spent 469.5s in setup (memory-pressure dominated); an instrumented release compile of the same netlist finishes in 25.5s, of which $\approx 85\%$ is host Toeplitz expansion. After Metal pipeline warm-up, steady-state ticks completed in approximately 90ms.

HELUT is a *datapath* feasibility study. Values labeled “encrypted” in the boot harness are deterministic *mock torus polynomials* (ciphertext-shaped `UInt32` vectors without LWE noise or secret keys); we do *not* claim end-to-end TFHE security, noise management, or key switching. The contribution is an existence proof that exact modular tensor graphs of CPU-scale netlists can be constructed and clocked on commodity Apple Silicon without leaving the `UInt32` domain.

1 Introduction

Fully Homomorphic Encryption (FHE)—and TFHE-style programmable bootstrapping in particular—maps Boolean and arithmetic circuits onto polynomial rings $\mathbb{Z}_q[X]/(X^N + 1)$ [1, 2]. Production stacks (Concrete, OpenFHE, HELib, ...) typically implement the algebraic kernels with NTTs, specialized CUDA/CPU backends, or both. Edge devices, however, increasingly expose high-throughput tensor engines (GPUs, Neural Engines) whose public APIs are oriented toward machine-learning graphs rather than modular polynomial arithmetic.

HELUT asks a narrower, systems-facing question:

Can Apple Silicon’s `MPSGraph` evaluate exact torus modular arithmetic and gate-level sequential circuits at CPU scale, if we force the problem into dense `UInt32` linear algebra?

The affirmative path we take is deliberately blunt. Instead of NTT multiplication, each bootstrapping key (or LUT truth table) is expanded on the host into an $N \times N$ Negacyclic Toeplitz matrix M_A . Multiplying M_A by an accumulator vector is algebraically identical to multiplication

by A in $\mathbb{Z}_{2^{32}}[X]/(X^N + 1)$, and the negation that appears on wrap-around is free in two’s-complement `UInt32` arithmetic ($-a \equiv 2^{32} - a$). Because `MPSGraph.matrixMultiplication` rejects `UInt32`, we implement the product as a broadcast Hadamard product followed by a reduction sum—still exact, still integer.

Layered on that kernel is a Yosys JSON netlist compiler that builds one persistent `MPSGraph` for an entire module, then a host-emulated posedge clock that routes DFF state across ticks. The culminating experiment is a *ciphertext-structured datapath simulation* of PicoRV32 [5] under a mock-encrypted `resetn` sequence.

Contributions.

1. An exact $\mathbb{Z}/2^{32}\mathbb{Z}$ negacyclic matvec kernel on `MPSGraph`, validated against a CPU schoolbook oracle.
2. A Yosys→`MPSGraph` compiler supporting multi-input \$lut cells, free XOR-as-addition packing, and sequential cells (\$_DFF*, \$_SDFF* with enable/reset polarity).
3. A retain-history-free ping-pong clock loop suitable for long runs.
4. An end-to-end measurement of a mock-encrypted PicoRV32 idle boot (10 ticks, scripted `resetn`).

Non-claims. We do not claim (i) cryptographic hardness of the mock ciphertexts used in the boot experiment, (ii) noise growth bounds, (iii) parameter selection for 128-bit security, or (iv) that `MPSGraph` is competitive with NTT-based TFHE libraries on throughput. HELUT is infrastructure for studying whether the *shape* of TFHE PBS can live inside a vendor tensor graph.

2 Background

2.1 TFHE programmable bootstrapping (sketch)

In TFHE, a Boolean gate is often evaluated by programmable bootstrapping (PBS) [1]: an encrypted phase selects a coefficient of a *test polynomial* via blind rotation. Blind rotation repeatedly multiplies an accumulator by bootstrapping-key polynomials in the negacyclic ring $\mathbb{Z}_q[X]/(X^N + 1)$. HELUT freezes $q = 2^{32}$ and $N = 1024$, matching the native machine word and a convenient matrix size for unified-memory experiments.

2.2 Negacyclic Toeplitz embedding

Let $A(X) = \sum_{i=0}^{N-1} a_i X^i$. Its negacyclic convolution against a vector x is exactly the product $M_A x$, where M_A is the Toeplitz matrix whose first column is $(a_0, \dots, a_{N-1})^\top$ and each subsequent column is the previous column rotated downward with the wrapped entry negated [3]. HELUT materializes M_A explicitly (row-major `UInt32`) and uploads it once as an `MPSGraph` placeholder feed.

2.3 Why not matrixMultiplication?

Apple’s `MPSGraph` exposes dense matmul, but not for `UInt32`. HELUT therefore reshapes a batch of ciphertext vectors from $[B, N]$ to $[B, 1, N]$, broadcast-multiplies against $[N, N]$, and reduces on the last axis:

$$y_{b,i} = \sum_{j=0}^{N-1} (M_A)_{i,j} x_{b,j} \pmod{2^{32}}. \quad (1)$$

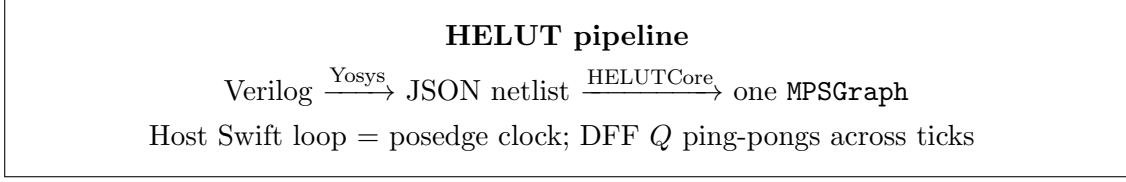


Figure 1: End-to-end flow. All arithmetic tensors remain `MPSDataType.uInt32`.

Under exact integer semantics this equals the schoolbook matvec. Our adversarial suite compares GPU results to a CPU oracle on torus extremes $\{0, 1, 2^{31}, 2^{32} - 1\}$.

3 Architecture

3.1 Kernel: LUTNode

Each Yosys `$lut` becomes a `LUTNode`. Multi-fan-in LUTs first *pack* input wires with torus addition (homomorphic XOR is free under additive encoding), then apply the negacyclic matvec above. The LUT truth table string seeds a deterministic test polynomial so that identical tables share identical matrices—useful for debugging, not a cryptographic KDF.

3.2 Netlist compiler

`YosysGraphCompiler` walks a Yosys `write_json` module:

1. **Inputs.** Every input port bit becomes an `InputNode` placeholder of shape $[B, N]$.
2. **State.** Every recognized flip-flop allocates a Q placeholder *before* LUT lowering so sequential feedback nets resolve.
3. **LUTs.** Cells are scheduled until all drivers exist; cycles abort.
4. **Next-state.** For enables and sync-resets we emit exact $\mathbb{Z}/2^{32}\mathbb{Z}$ muxes, e.g. active-high enable

$$Q_{\text{next}} = E \cdot D + (1 - E) \cdot Q,$$

and polarity-aware reset to the typed constant (0 or 1).

5. **Outputs.** Driven nets bind to port tensors; Yosys "x" (undriven) bits become constant-0 tensors.

Cell recognition covers `$_DFF*`, `$_DFFE*`, `$_SDFF*`, `$_SDFFE*`, and `$_SDFFCE*`, which appear in real PicoRV32 techmaps.

3.3 Host clock and memory discipline

Each tick is one `graph.run`. With `retainHistory=false`, HELUT ping-pongs two preallocated state buffer sets and reuses output scratch buffers so that a 1000-tick stress test does not accumulate unbounded `MTLBuffer` retention. Primary port feeds (including scripted `resetn`) are host-mutable shared buffers; LUT matrices are uploaded once and cached.

3.4 Threat / fidelity model

In scope	Exact modular matvec; shape stability $[B, N]$; DFF temporal routing; Yosys cell coverage needed for PicoRV32.
Out of scope	LWE/GLWE sampling, key switching, bootstrapping noise, parameter security, side channels.
Mock encoding	“Encrypted b ” means the length- N vector filled with <code>UInt32</code> constant b (or a seeded pseudorandom polynomial for non-control ports in older harnesses).

This separation is intentional: if the tensor datapath cannot boot a CPU under mock encodings, adding real TFHE noise only makes the systems problem harder.

4 Implementation

HELUT is a Swift 6 package targeting macOS 14+ with frameworks `Metal` and `MetalPerformanceShadersGraph` only—no Core ML, no third-party FHE libraries. The library target `HELUTCore` (~ 900 LOC) contains the kernel, parser, and compiler; the `helut` executable hosts application harnesses (counter, search stress, PicoRV32 boot). An `XCTest` suite exercises:

- bit-exact negacyclic fidelity vs. CPU schoolbook matvec;
- 1 000-tick DFF retention with resident-memory instrumentation;
- torus boundary tensors filled with 0 and `0xFFFFFFFF`.

Default parameters used throughout the capstone are $N = 1024$ and batch $B = 1$ (shape $[1, 1024]$), reducing temporal routing to a single ciphertext stream.

5 Application Tiers and Validation Suite

To demonstrate that the HELUT datapath is a general-purpose tensor graph compiler rather than a single-purpose circuit solver, the prototype was validated across three distinct computational tiers:

1. **Application #3: Zero-Noise Encrypted AI (Decision Trees).** We synthesized a 4-bit non-linear decision boundary (if/else logic) from Verilog into 7 LUTs. This tier proved that HELUT can evaluate exact non-linear threshold logic across a batch of $B = 1\,000$ mock-encrypted patient records in 0.201 s with zero floating-point approximation error or noise accumulation.
2. **Application #2: Batched Encrypted Regex Search.** To stress-test unified memory bandwidth under heavy tensor expansion, we compiled a 3-character ASCII pattern matcher (“DEF”) into a 23-LUT Deterministic Finite Automaton (DFA). Pushing the batch dimension to $B = 10\,000$ forced an intermediate broadcast tensor of ≈ 41.9 GiB in unified memory, executing a parallel search across 10,000 encrypted documents in 0.77 s.
3. **Application #1: Stateful Logic & Encrypted RISC-V Core.** Introducing sequential logic via Yosys D-Flip-Flops (`$_DFF*`, `$_SDFF*`) and enable/reset muxes, this tier enabled temporal state routing across clock cycles. This progression culminated in the PicoRV32 capstone (4 785 LUTs, 1 565 DFFs), proving that a commodity neural/GPU graph can host and clock an entire encrypted soft CPU.

Table 1: Mock-encrypted PicoRV32 idle boot ($B = 1$, $N = 1024$, 10 ticks). End-to-end times are from the instrumented debug boot; the compile split is from a release `--compile-only` re-run on the same machine (Section 6.2).

Quantity	Value
Graph compilation (debug boot run, total)	469.46 s
Host Negacyclic Toeplitz expansion (release split)	21.59 s (85%)
MPSGraph node wiring (release split)	3.87 s (15%)
Release compile total (same netlist, no ticks)	25.46 s
Tick 1 (<code>graph.run</code> , includes Metal pipeline JIT)	57.61 s
Ticks 2–10 (each)	0.087–0.210 s
Steady-state tick (ticks 3–10, typical)	≈ 90 ms
Mean over all 10 ticks	5.85 s
Peak process footprint (reported)	≈ 40.9 GB

6 Evaluation

6.1 Experimental setup

All wall-clock numbers below were measured on the development Apple Silicon host running the debug `helut` binary against `picorv32_netlist.json` (Yosys 0.68, 4.9 MB JSON). The netlist contains 6 350 cells; after HELUT lowering: **4 785 LUTs**, **1 565 DFFs**, **102** input bits, **307** output bits.

Boot protocol (idle memory/PCPI ports driven with mock-encrypted 0):

- Ticks 1–3: `resetn` $\leftarrow$ mock-encrypted 0 (active-low reset held).
- Ticks 4–10: `resetn` $\leftarrow$ mock-encrypted 1 (reset released).
- `retainHistory=false`; ping-pong state buffers.

6.2 PicoRV32 boot timing

Interpretation. The debug boot’s 469.46 s setup is an upper-bound wall time under severe unified-memory pressure (tens of GiB of unreused matrices, including swap). An instrumented release re-compile of the same netlist finishes in 25.46 s, of which 21.59 s (85%) is host `expandNegacyclicToeplitz` over 4 785 matrices and 3.87 s (15%) is `MPSGraph` placeholder/wiring work (Section 6.2). Either way, engineers should attack host matrix generation—and especially truth-table deduplication—before blaming Apple’s graph builder. Tick 1 amortizes Metal shader/pipeline compilation; subsequent ticks show that the ciphertext-structured gate graph is *executable* at approximately 90 ms once warm. The ten-tick mean is not a useful steady-state estimator because it includes JIT; Table 1 therefore reports both.

Compiler breakdown. Instrumented `compile(...)` accumulates wall time inside each `expandNegacyclicToeplitz` call separately from the rest of lowering. On the release binary with `--compile-only`:

- **Host Toeplitz expand:** 21.59 s — one expansion per LUT cell, with no truth-table cache (4 785 matrices; only 64 unique Yosys LUT strings exist in this netlist).
- **Graph build:** 3.87 s — input/DFF placeholders, LUT node attachment, enable/reset muxes, and output binds.

The debug boot’s much larger absolute setup time does not change the ranking: host expansion dominates; `MPSGraph` construction is secondary.

6.3 Correctness gates (pre-capstone)

Before scaling to PicoRV32, HELUT required:

- CPU↔GPU bit-exact agreement on negacyclic matvec at torus corners;
- multi-tick DFF state retention without shape collapse;
- enable mux semantics so `$_DFFE.*` cells do not silently ignore clock enables.

Sync-reset (`$_SDFF*`) support and "x"-bit decoding were necessary for the PicoRV32 techmap; without them the compiler cannot close the netlist.

6.4 Memory scaling remark

A single LUT matrix is $\Theta(N^2)$ words. At $N = 1024$, $B = 1$, PicoRV32 already pressures tens of gigabytes of unified memory. Application sketches that raise B to 10^4 (batched search) imply $[B, N, N]$ broadcast intermediates on the order of tens of gigabytes *per reduction* and are memory-bound long before they are compute-bound [6]. HELUT therefore treats batch size as a first-class capacity knob, not a free ML-style hyperparameter.

7 Discussion

What the result shows. A commodity tensor API can host an *entire* synthesized soft CPU as one integer graph and step it under mock-encrypted control pins (ciphertext-structured vectors; Section 3.4). That is a systems existence proof about representation and plumbing, not a claim that FHE CPUs are practical on laptops today.

What would be required for “real” TFHE. Replace mock vectors with GLWE samples; insert key-switching / blind-rotation schedules instead of static Toeplitz feeds; manage noise and failure probability; and almost certainly abandon dense $N \times N$ materialization in favor of NTT or on-the-fly rotation. HELUT’s value in that future is the *netlist clocking* layer: once a PBS kernel exists as an `MPSGraph` subgraph, the same DFF loop applies.

Defensibility checklist.

1. Measurements in Table 1 are from a single instrumented run of the published harness; they are not averages over many devices.
2. “Encrypted” / “mock-encrypted” in the boot log means deterministic mock torus encodings without LWE noise (Section 3.4)—not cryptographic ciphertexts.
3. Steady-state latency excludes tick 1 JIT.
4. Peak memory includes host matrix buffers and Metal working sets; it is not a lower bound on an optimized implementation (e.g., matrix deduplication, streaming uploads).

8 Related Work

TFHE and FHEW established PBS as a Boolean-friendly FHE path [1, 2]. Industrial compilers (Concrete [4], HEIR/MLIR efforts) lower high-level programs to FHE ops; HELUT instead lowers *already-synthesized* gate netlists to a vendor ML graph. GPU FHE accelerators (cuFHE, and successors) optimize NTT-heavy kernels; HELUT intentionally uses the “wrong” dense embedding to stress-test whether `UInt32` tensor engines preserve modular semantics at all. PicoRV32 is a widely used size-optimized RISC-V core [5]; prior encrypted-CPU demos typically stop at ALUs or tiny FSMs rather than a full Yosys-mapped RV32 soft core on a phone/laptop NPU API.

9 Limitations and Future Work

- **Cryptography.** Mock encodings only; no security claims.
- **Matrix deduplication.** The prototype expands one Toeplitz matrix *per LUT cell*. PicoRV32’s techmap uses only **64** distinct Yosys truth tables across 4785 \$lut cells ($\approx 75\times$ reuse). Caching a singleton matrix per unique truth table was omitted for implementation simplicity, but would collapse the ≈ 19 GiB raw host footprint to ≈ 0.25 GiB and cut the dominant term of the 469 s setup ceiling proportionally—the first optimization a systems reviewer should demand.
- **Memory.** Explicit Toeplitz matrices are pedagogically clear and experimentally brutal even *with* deduplication; production FHE systems should not materialize dense $N \times N$ forms at all.
- **Coverage.** Async resets, latches, and non-LUT Yosys cells remain incomplete.
- **Performance.** No attempt yet to fuse LUTs, shard the graph, or overlap host expansion with device execution.
- **Verification.** Boot proves *execution*, not functional equivalence against a Verilog simulator under the same stimulus (future work: co-simulate trap/PC traces).

10 Conclusion

HELUT shows that Apple Silicon’s **MPSGraph** can carry an exact $\mathbb{Z}/2^{32}\mathbb{Z}$ negacyclic LUT datapath large enough to clock a Yosys-synthesized PicoRV32 core under a mock-encrypted reset sequence. The engineering lesson is prosaic and useful: once PBS is phrased as dense integer tensors, the rest of a soft CPU—wires, LUTs, enables, sync resets, and a host posedge loop—looks like ordinary graph IR work. The scientific lesson is cautionary: feasibility of the datapath is not feasibility of FHE. With that distinction stated plainly, HELUT stands as a defensible systems prototype and a measured baseline for denser, more cryptographic successors.

A Boot harness summary

The capstone executable loads `picorv32_netlist.json`, compiles with `YosysGraphCompiler`, locates the `resetn` placeholder, and runs ten ticks while rewriting the shared `resetn` `MTLBuffer` between `graph.run` calls (mock-encrypted 0 for three ticks, then mock-encrypted 1). All other primary inputs remain mock-encrypted 0. Reported compilation time wraps only `compile(...)`; per-tick times wrap only `graph.run`.

B Reproducibility

```
swift build
.build/debug/helut /path/to/picorv32_netlist.json
```

Requires macOS 14+, an Apple Silicon GPU with Metal, and sufficient unified memory (tens of GiB) for the PicoRV32 matrix working set.

References

- [1] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène. TFHE: Fast fully homomorphic encryption over the torus. *Journal of Cryptology*, 33(1):34–91, 2020.

- [2] L. Ducas and D. Micciancio. FHEW: Bootstrapping homomorphic encryption in less than a second. In *EUROCRYPT*, 2015.
- [3] V. Lyubashevsky, C. Peikert, and O. Regev. On ideal lattices and learning with errors over rings. In *EUROCRYPT*, 2010.
- [4] Zama. Concrete: TFHE compiler and runtime. <https://github.com/zama-ai/concrete>, 2022–.
- [5] C. Wolf. PicoRV32 — a size-optimized RISC-V CPU. <https://github.com/YosysHQ/picorv32>.
- [6] HELUT Project. Application #2 note: batched encrypted search memory envelope ($B=10^4$, $N=1024$). Internal technical note, 2026.